GenAI Code Comparison - Sudoku Solver

GenAI programs used:

- Google Gemini: I used the Gemini chat interface.
- Anthropic Claude: I used the Claude chat interface.
- I gave both tools the same assignment prompt to generate their C++ programs.

Prompt: I combined the assignment instructions into a shorter prompt in ChatGPT, then gave that same prompt to both tools along with the puzzle1 puzzle and its solution PDF.

> Task: Write a C++ program that solves Sudoku puzzles. It must be recursive and object-oriented, using classes and objects rather than standalone functions.
> What Sudoku is:
>
> - A 9×9 grid divided into nine 3×3 blocks.
> - Every cell gets one digit from 1 to 9.
> - Each digit appears exactly once in every row, every column, and every block.
> - Published puzzles usually have one solution and need at least 17 filled cells. The number of filled cells doesn’t say much about how hard a puzzle is.
>
> The problem to solve: Given a partly filled puzzle, find a solution, or show that none exists.
> Required approach: brute-force depth-first search with backtracking, not logic-based solving.
>
> 1. Find an empty cell.

2. List the digits that could go there, meaning those not already in the same row, column, or block.
3. If there are no candidates, an earlier choice was wrong, so return “no solution.”
4. Otherwise, place the first candidate and recursively try to solve the rest of the board.
5. If that fails, try the next candidate, and so on.
6. Stop when the board is full (all 81 cells), or when every candidate fails, which means no solution on this path.

- Because this tries every possibility, it will always find a solution if one exists. It should run in a few tenths of a second.

Input:

- Five files, puzzle1.txt through puzzle5.txt.
- Given digits are filled in, and an underscore (_) marks a blank cell.

Output for each puzzle:

- The file name (for example, puzzle1.txt).
- The original puzzle as read from the file.
- The solution, or “No solution found” if there isn’t one.
- If a puzzle has more than one solution, print all of them.

Debugging aid: solution1.txt contains the correct solution to puzzle1.txt.

Correctness

- Gemini uses recursive row-and-column traversal, checks the Sudoku rules, and prints each solution it finds. One issue is that it does not check whether the original clues contradict each other before solving, so a completed but invalid grid could be printed as a solution. Its parser also skips unexpected nonblank characters instead of rejecting them.

- Claude also uses recursive backtracking and prints every solution it finds. Its givenAreValid() method checks for conflicting clues before the search starts. The parser rejects files with fewer than 81 recognized symbols, but it accepts extra symbolss after the first 81 and skips unexpected characters. I could not test either program with the five official puzzle files because they were not available here.

Execution Time

- Both programs use depth-first search with backtracking. In the worst case, the search grows exponentially with the number of empty cells, E (a loose upper bound is O(9^E)). Sudoku-rule checks prune many branches, so the actual search can be much smaller. Gemini checks whether each candidate is legal by checking its row, column, and 3×3 block, then moves directly to the next cell. Claude scans the board for the next empty cell on every recursive call, which can mean up to 81 checks per call, and it builds a candidate vector.

Space complexity

- Gemini uses two fixed 9×9 integer arrays and a recursion stack that can reach at most 81 calls. It prints each solution as soon as it finds it instead of saving it, so its extra space for the search is O(E), where E is the number of empty cells. Claude stores a 9×9 board, uses a recursion stack of depth at most E, and keeps all completed boards in a vector. If it finds S solutions, that vector takes O(81S) space, making Claude's memory use grow when a puzzle has many solutions.

Maintainability

- Gemini's implementation is smaller and uses one class, so it is fairly easy to follow, but that class handles file input, validation, solving, and output all together. Claude splits those jobs across Board, Solver Puzzlerunner, and SodokuApp. That takes more code, but it also makes each class's role easier to understand and update. Both original programs include some comments.

Selection and justification

- I chose Claude because its separate classes make the program easier to maintain, and its check for conflicting starting clues is an important correctness improvement.

- Gemini does have a small memory advantage because it prints solutions instead of saving them all. I used that approach in my revised version.

Changes made and effects

- Correctness: I updated the Board parser so it accepts exactly 81 digits or underscores, skips whitespace, and rejects unexpected characters or extra cells. If reading the file fails, it leaves the existing board unchanged. Before solving, the program also checks that the original clues do not contradict each other.
- Execution time: The solver precomputes the coordinates of empty cells once and indexes that list in recursive calls. This removes repeated 9×9 scans for the next empty position. Candidate validation still checks the row, column, and box.
    - no unmeasured speedup is claimed
- Space complexity: Now, the program prints each completed board right away, so it no longer stores Claude's vector of solutions, which could take O(81S) space for S solutions. The list of blank-cell coordinates takes O(E) space, and the recursive call stack also uses O(E) space.
- Maintainability: I kept the separate Board, Solver, PuzzleRunner, and SudokuApp classes. The program reports input problems clearly, and the source prologue and comments explain how it works and where the original code came from.

Testing and limits

- I compiled the revised program in the available environment with g++ -std=c++17 -O2 -Wall -Wextra -pedantic.
- The program compiled successfully without errors, and I ran the executable using ./Assignment4.exe.
- Results:

Puzzle File	Result
puzzle1.txt	1 solution found
puzzle2.txt	1 solution found

Puzzle File	Result
puzzle3.txt	1 solution found
puzzle4.txt	No solution found
puzzle5.txt	2 solutions found

- The program correctly shows each original puzzle, printed the completed solutions, and reported when no solution existed. Puzzle 5 was really useful because it confirmed that the recursive backtracking algorithm continues searching after finding its first solution rather than stopping immediately. The tests with all the puzzles confirmed that my edited program handles puzzles with one solution, no solution, and multiple solutions. It also showed that the changes I made did not prevent the program from solving the provided puzzles correctly.

Required raw GenAI code:

Appendix A: Original Gemini C++ source:

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    static const int GRID_SIZE = 9;
    static const int SUBGRID_SIZE = 3;

    int grid[GRID_SIZE][GRID_SIZE];
    int initialGrid[GRID_SIZE][GRID_SIZE];
    std::string filename;
    int solutionCount;

    // Checks if placing num at grid[row][col] is valid
    bool isValid(int row, int col, int num) const {
        for (int i = 0; i < GRID_SIZE; ++i) {
```

```
            // Check row and column constraints
            if (grid[row][i] == num || grid[i][col] == num) {
                return false;
            }
        }

        // Check 3x3 block constraint
        int startRow = (row / SUBGRID_SIZE) * SUBGRID_SIZE;
        int startCol = (col / SUBGRID_SIZE) * SUBGRID_SIZE;

        for (int i = 0; i < SUBGRID_SIZE; ++i) {
            for (int j = 0; j < SUBGRID_SIZE; ++j) {
                if (grid[startRow + i][startCol + j] == num)
{
                    return false;
                }
            }
        }

        return true;
    }

    // Print the grid
    void printGrid(const int targetGrid[GRID_SIZE][GRID_SIZ
E]) const {
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                if (targetGrid[r][c] == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << targetGrid[r][c] << " ";
                }
            }
            std::cout << "\n";
        }
    }
```

```
    // Recursive depth-first search solver
    void solveRecursive(int row, int col) {
        // If we reached past the last row, a complete valid
solution is found
        if (row == GRID_SIZE) {
            solutionCount++;
            std::cout << "\nSolution #" << solutionCount <<
":\n";
            printGrid(grid);
            return;
        }

        // Determine next cell indices
        int nextRow = (col == GRID_SIZE - 1) ? row + 1 : row;
        int nextCol = (col == GRID_SIZE - 1) ? 0 : col + 1;

        // Skip non-empty cells
        if (grid[row][col] != 0) {
            solveRecursive(nextRow, nextCol);
            return;
        }

        // Try candidate numbers 1 through 9
        for (int num = 1; num <= GRID_SIZE; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;

                // Recurse to solve remaining cells
                solveRecursive(nextRow, nextCol);

                // Backtrack
                grid[row][col] = 0;
            }
        }
    }
```

```
public:
    explicit SudokuSolver(const std::string& file) : filename
(file), solutionCount(0) {
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                grid[r][c] = 0;
                initialGrid[r][c] = 0;
            }
        }
    }

    // Loads puzzle from file
    bool loadPuzzle() {
        std::ifstream file(filename);
        if (!file.is_open()) {
            std::cerr << "Error opening file: " << filename <
< "\n";
            return false;
        }

        char ch;
        for (int r = 0; r < GRID_SIZE; ++r) {
            for (int c = 0; c < GRID_SIZE; ++c) {
                if (!(file >> ch)) {
                    std::cerr << "Invalid file format in " <<
filename << "\n";
                    return false;
                }
                if (ch >= '1' && ch <= '9') {
                    grid[r][c] = ch - '0';
                    initialGrid[r][c] = ch - '0';
                } else if (ch == '_') {
                    grid[r][c] = 0;
                    initialGrid[r][c] = 0;
                } else {
```

```
                    // Ignore extra characters or line breaks
automatically handled by >> operator
                    c--;
                }
            }
        }
        file.close();
        return true;
    }

    // Orchestrates solving and output display
    void solve() {
        std::cout << "========================================
=\n";
        std::cout << "File: " << filename << "\n";
        std::cout << "Initial Puzzle:\n";
        printGrid(initialGrid);

        solutionCount = 0;
        solveRecursive(0, 0);

        if (solutionCount == 0) {
            std::cout << "\nNo solution found\n";
        } else {
            std::cout << "\nSolutions found: " << solutionCou
nt << "\n";
        }
        std::cout << "========================================
=\n\n";
    }
};

int main() {
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
```

```
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto& file : puzzleFiles) {
        SudokuSolver solver(file);
        if (solver.loadPuzzle()) {
            solver.solve();
        }
    }

    return 0;
}
```

Appendix B: Original Claude C++ source:

```
// sudoku.cpp
// Recursive, object-oriented Sudoku solver using depth-first
search
// with backtracking. Finds and prints ALL solutions for each
puzzle file.
//
// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:    ./sudoku                        (solves puzzle1.txt
.. puzzle5.txt)
//         ./sudoku myPuzzle.txt other.txt (solves the files
you list)

#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
```

```
// ------------------------------------------------------------
--
// Board: holds a 9x9 grid and knows the Sudoku rules.
// ------------------------------------------------------------
--
class Board {
public:
    static const int SIZE = 9;
    static const int BOX = 3;
    static const int EMPTY = 0;

    Board() {
        for (auto& row : cells) row.fill(EMPTY);
    }

    // Reads a puzzle file. Digits 1-9 are givens; '_' (or
'0' / '.') is blank.
    bool loadFromFile(const std::string& fileName) {
        std::ifstream in(fileName);
        if (!in) return false;

        int count = 0;
        char ch;
        while (count < SIZE * SIZE && in >> ch) {
            int r = count / SIZE;
            int c = count % SIZE;
            if (ch >= '1' && ch <= '9') {
                cells[r][c] = ch - '0';
            } else if (ch == '_' || ch == '0' || ch == '.') {
                cells[r][c] = EMPTY;
            } else {
                continue; // ignore any other character
            }
            ++count;
        }
        return count == SIZE * SIZE;
```

```
    }

    int get(int row, int col) const { return cells[row][col];
}
    void set(int row, int col, int value) { cells[row][col] =
value; }
    void clear(int row, int col) { cells[row][col] = EMPTY; }

    // True if 'value' does not already appear in the row, co
lumn, or block.
    bool canPlace(int row, int col, int value) const {
        for (int i = 0; i < SIZE; ++i) {
            if (cells[row][i] == value) return false;
            if (cells[i][col] == value) return false;
        }
        int startRow = (row / BOX) * BOX;
        int startCol = (col / BOX) * BOX;
        for (int r = startRow; r < startRow + BOX; ++r)
            for (int c = startCol; c < startCol + BOX; ++c)
                if (cells[r][c] == value) return false;
        return true;
    }

    // List of digits that could legally go in an empty cell.
    std::vector<int> candidates(int row, int col) const {
        std::vector<int> result;
        for (int v = 1; v <= SIZE; ++v)
            if (canPlace(row, col, v)) result.push_back(v);
        return result;
    }

    // Finds the first empty cell. Returns false if the board
is full.
    bool findEmpty(int& row, int& col) const {
        for (row = 0; row < SIZE; ++row)
            for (col = 0; col < SIZE; ++col)
```

```
                if (cells[row][col] == EMPTY) return true;
        return false;
    }

    // Makes sure the givens do not already break a rule.
    bool givensAreValid() {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                int v = cells[r][c];
                if (v == EMPTY) continue;
                cells[r][c] = EMPTY;              // temporar
ily remove it
                bool ok = canPlace(r, c, v);
                cells[r][c] = v;                  // put it b
ack
                if (!ok) return false;
            }
        }
        return true;
    }

    void print(std::ostream& out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (cells[r][c] == EMPTY) out << "_ ";
                else out << cells[r][c] << ' ';
            }
            out << '\n';
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> cells;
};

// ----------------------------------------------------------
```

```
--
// Solver: recursive depth-first search with backtracking.
// Collects every solution it finds.
// ------------------------------------------------------------
--
class Solver {
public:
    explicit Solver(const Board& start) : board(start) {}

    void solveAll() {
        solutions.clear();
        if (!board.givensAreValid()) return; // contradictory
puzzle
        solve();
    }

    const std::vector<Board>& getSolutions() const { return s
olutions; }

private:
    Board board;
    std::vector<Board> solutions;

    // Recursive step: fill one empty cell, then recurse on t
he rest.
    void solve() {
        int row, col;
        if (!board.findEmpty(row, col)) {
            // All 81 cells filled: record a solution, then k
eep searching
            // so that every solution is found.
            solutions.push_back(board);
            return;
        }

        // If there are no candidates, this loop does nothing
```

```
and we
        // return, which backtracks to the previous cell.
        for (int value : board.candidates(row, col)) {
            board.set(row, col, value);
            solve();                // recursive call with k+
1 cells filled
            board.clear(row, col);  // undo (backtrack)
        }
    }
};

// ------------------------------------------------------------
--
// PuzzleRunner: handles one puzzle file from start to finis
h.
// ------------------------------------------------------------
--
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string& file) : fileName
(file) {}

    void run(std::ostream& out) const {
        out << "==============================\n";
        out << "Puzzle file: " << fileName << "\n";
        out << "==============================\n";

        Board puzzle;
        if (!puzzle.loadFromFile(fileName)) {
            out << "Could not read a valid 9x9 puzzle from "
<< fileName << "\n\n";
            return;
        }

        out << "Puzzle:\n";
        puzzle.print(out);
```

```
        out << '\n';

        Solver solver(puzzle);
        solver.solveAll();
        const std::vector<Board>& found = solver.getSolutions
();

        if (found.empty()) {
            out << "No solution found\n\n";
            return;
        }

        for (size_t i = 0; i < found.size(); ++i) {
            out << "Solution " << (i + 1) << ":\n";
            found[i].print(out);
            out << '\n';
        }
        out << "Solutions found: " << found.size() << "\n\n";
    }

private:
    std::string fileName;
};

// ----------------------------------------------------------
--
// SudokuApp: runs every puzzle file.
// ----------------------------------------------------------
--
class SudokuApp {
public:
    explicit SudokuApp(const std::vector<std::string>& files)
: fileNames(files) {}

    void run() const {
        for (const std::string& name : fileNames) {
```

```
            PuzzleRunner runner(name);
            runner.run(std::cout);
        }
    }

private:
    std::vector<std::string> fileNames;
};

int main(int argc, char* argv[]) {
    std::vector<std::string> files;
    if (argc > 1) {
        for (int i = 1; i < argc; ++i) files.push_back(argv
[i]);
    } else {
        for (int i = 1; i <= 5; ++i)
            files.push_back("puzzle" + std::to_string(i) + ".
txt");
    }

    SudokuApp app(files);
    app.run();
    return 0;
}
```